

Feature selection

- Muhim bo'lgan featurelarni tanlab olish jarayoni - FS
 - Modelning:
 - Aniqlogini oshiradi
 - Training tezligini oshiradi
 - Interpretability ni oshiradi
- ↓
Featurelarni oson tutkuvchi

Feature selection: Methods

- Filter
- Wrapper
- Embedded

FS: Filter method

- Statistical qiymatlariga ko'ra featurelarni tanlash
- Feature ni qanchalik target qiymatga bog'liqligiga ko'ra tanlaydi
- 1: Strong relationship
- 0: Independent (no relationship)
- Pearson correlation coefficient
- Training qilishdan avval / PreProcessing va FE dan keyin
- Methodlar: Correlation, Variance Threshold, ANOVA
- Qanday ishlaydi:
 - Juda yuqori correlated featurelarni tashlab yuboradi (multicollinearity)
 - Juda past correlated featurelarni tashlab yuboradi (low variance)
- Plus tomonlari (+)
 - Tez ishlaydi (Algorithm, training kerak emas)
 - High-dimensional datasetlar bilan yaxshi ishlaydi
 - Overfittingni kamaytiradi
- Minus tomonlari (-)
 - Barcha featurelarni alohida-alohida baxolanadi
 - Kafolat yo'q

FS: Filter Method (Analogy)

Filter Methods:

Correlation

```

import numpy as np
corr_matrix = df.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [col for col in upper.columns if any(upper[col] > 0.8)]
df_filtered = df.drop(columns=to_drop)
print("In Original shape:", df.shape)
print("Shape after dropping correlated features:", df_filtered.shape)

```

More analysis

```

import seaborn as sns
import matplotlib.pyplot as plt
corr_matrix = df.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
high_corr_pairs = []
for col in upper.columns:
    for row in upper.index:
        if upper.loc[row, col] is not np.nan and upper.loc[row, col] > 0.8:
            high_corr_pairs.append((row, col, round(upper.loc[row, col], 2)))
corr_table = pd.DataFrame(high_corr_pairs, columns=['Feature 1', 'Feature 2', 'Correlation'])
print("Highly correlated feature pairs (correlation > 0.8):")
print(corr_table)

```

```

df_filtered = df.drop(columns=to_drop)
print("In Original shape:", df.shape)
print("Shape after dropping correlated features:", df_filtered.shape)

```


Date.

No.

```

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", cbar=True)
plt.title("Correlation Matrix Heatmap")
plt.show()

# Plotly express
import Plotly.express as px

# Create a long-form dataframe for Plotly
corr_long = corr_matrix.reset_index().melt(id_vars='index')
corr_long.columns = ['Feature 1', 'Feature 2', 'Correlation']

# Interactive heatmap with Plotly Express
fig = px.imshow(
    corr_matrix, text_auto='.2f', # show correlation values
    aspect="auto", color_continuous_scale="RdBu_r", (red-blue reserved)
    zmin=-1, zmax=1,
    title="Correlation Matrix Heatmap (Interactive)"
)

fig.update_layout(width=900, height=800,
                  xaxis_title="Features",
                  yaxis_title="Features")

fig.show()

# Correlation matrix using pandas
corr_matrix = df.corr().abs()
print(corr_matrix)

```


Low Variance

from sklearn.feature_selection import VarianceThreshold

Set threshold for variance (eg., 0.01)

threshold = 0.01

Initialize VarianceThreshold

selector = VarianceThreshold(threshold=threshold)

Fit on the dataset

selector.fit(df_filtered)

Features with Low Variance

low_variance_features = df_filtered.columns[~selector.get_support()]

print("Features with low variance (to drop):", list(low_variance_features))

Drop Low-variance features

df_low_variance_filtered = df_filtered.drop(columns=low_variance_features)

print("In Original shape:", df_filtered.shape)

print("Shape after dropping low-variance features:", df_low_variance_filtered.shape)

low_variance_features = df_filtered.columns[~selector.get_support()]

print("Features with low variance (to drop):", list(low_variance_features))

Plotly express

numeric_cols = df.select_dtypes(include=[np.number]).columns

variances = df[numeric_cols].var()

threshold = 0.01

low_variance_features = variances[variances < threshold].index.tolist()

var_df = pd.DataFrame({
 'Feature': variances.index,
 'Variance': variances.values,
 'Low Variances': ['Yes' if f in low_variance_features
 else 'No' for f in variances.index]})

fig = px.bar(var_df,
 x='Feature', color='Low Variances',
 y='Variance', color_discrete_map={'Yes': 'red', 'No': 'blue'},
 text='Variance', title='Feature Variance (Low Variance Highlighted in Red)')

fig.update_layout(
 xaxis_ticangle=-45,
 width=1000,
 height=600)

fig.show()

df_filtered.info()

Bundan keyin ham bir featuralarni
 tekshirib qarashini drop qilishimiz kerak!

FS : Wrapper Methods

- ML model training qilgandan keyin featurelarni baralash
- Qandalik model accuracyga kisha qoshganiga ko'ra tanlaydi (relative contributions)
- Methods: RFE, Forward Selection, Backward Elimination
- Qanday ishlaydi:
 - Turli xil subsetting bilan ko'p marta training qiladi
 - Eng yuqori natija bergan subsetlar eng yaxshi featureslar sifatida qabul qiladi
- Plus tomonlari (+)
 - Featurelarni alohidadan ko'ra guruh bilan yaxshi natija berishni xisobga oladi
 - Odatda fitterdan ko'ra yaxshiroq natija beradi
- Minus tomonlari (-)
 - Juda ko'p vaqt oladi
 - Overfitga moyil

FS : Wrapper Methods (Amaliy)

Recursive Feature Elimination

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression

X = df.drop('x6', axis=1)
y = df['x6']
```

```
model = LinearRegression()
rfe = RFE(model, n_features_to_select=5)
rfe.fit(X, y)
selected_features = X.columns[rfe.support_]
print("Selected features:", selected_features)
```


FS: Embedded Methods

- ML model training davomida featurelarni tanlash usuli
- Ganchalik impurity kamaytirganiga ko'ra aniqladi (%) , total = 100%
- Methods: Lasso Regression, Gradient Boosting, Random Forest
- Qanday ishlaydi:
 - Model featurelarni muhimligiga ko'ra belgilaydi (feature importance) va impurityni ganchalik kamaytirganiga ko'ra aniqlaydi (aniqlanadi)
 - Nolg'a teng yoki juda past muhim belgan featurelarni tashlab yuboriladi
- impurity $\Rightarrow$ tartibsizlik (datasetni featurelari tartibsiz holati)
- Plus tomonlari (+)
 - Qulay, training davomida tutilanadi
 - Featurelarni birgalikda muhimligini e'tiborga oladi
 - Katta datasetlar bilan yaxshi ishlaydi
- Minus tomonlari (-)
 - Interpretability $\rightarrow$ qiyin (ozining tushunishimiz)
 - Scaling uchun ba'zi methodlar sensitive (balance bitmay qoladi)

FS: Embedded Methods (Amaliy)

Embedded Methods $\Rightarrow$ For Linear models

from sklearn.linear_model import LassoCV

X = df.drop('target', axis = 1)

y = df['target']

lasso = LassoCV(cv = 5, random_state = 42).fit(X, y)

importance = np.abs(lasso.coef_)

selected_features = X.columns[importance > 0]

print("Selected features using Lasso(non-zero coefficients):")

print(selected_features.tolist())

Sagat muhim featurelarni saqlab qolish uchun

percentile_threshold = np.percentile(importance, 75) # top 25% features

top_features = X.columns[importance >= percentile_threshold]

print("in Top 25% important features based on Lasso coefficients:")

print(top_features.tolist())

Table

```
feat_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance })
```

```
feat_df = feat_df.sort_values(by='Importance', ascending=False)
```

Mark top 25% most important feature

```
percentile_threshold = np.percentile(importance, 65)
```

```
feat_df['Top 25%'] = np.where(feat_df['Importance'] >= percentile_threshold, 'Yes', 'No')
```

Show table

```
print("Feature Importance Tables:")
print(feat_df)
```

```
fig = px.bar(feat_df,
             x='Feature', y='Importance',
             color='Top 25%', color_discrete_map={'Yes': 'red', 'No': 'blue'},
             text='Importance', title='Lasso Feature Importance')
```

```
fig.update_layout(xaxis_ticangle=-45,
                  width=1000, height=600)
```

fig.show()

Feature Selection (FE): Xulosa

• **Filter:** ⇒ Trainingdan oldin

- High-dimensional dataset
- Tezkor feature selection
- Model training siz

• **Wrapper:** ⇒ Trainingdan keyin

- Kichik yoki o'rtacha dataset bo'lsa
- Eng yuqori natija kerak bo'lsa
- Ko'p vaqt bo'lsa

• **Embedded:** ⇒ Training bilan birga

- Avtomatik feature selection
- O'rtacha dataset
- Xar bir algorithm uchun maxsus featurelar kerak bo'lsa

High-dimensional ⇒ columns va rows lar bir-biriga yaqin, ya'ni
1000 columns / 500 rows yoki 1000 columns / 1000 rows
2/1 1/1

Low-dimensional